5.1.1 How ACLs work

As shown in the previous section, Access Control Lists attach permissions directly to the resource they protect. This could be a file, a database record, a network share, a cloud storage object, or even a REST API endpoint. The ACL for ACME Corp's Marketing Campaign Plans folder, for instance, lists specific principals—Alice, Bob, and the Marketing team—alongside the actions each is allowed to perform. When someone attempts to access the folder, the system simply checks the list.

Here's how it works in practice: imagine Bob from the Product team tries to open a folder containing marketing plans. The system locates the ACL attached to the folder and checks for an entry for Bob. If the list shows that Bob is allowed to read and write, the action proceeds. If there's no matching entry or the entry denies the requested action, access is blocked. For example, in figure 5.1, Bob is allowed to read but not write; there's an entry for Bob, but if he tries to do anything other than read the folder, the action will be denied.

Similarly, imagine that Doug, a member of the Marketing team, tries to access a document in the folder. Even though he is not explicitly mentioned, his membership in a group, the Marketing team, allows him to read or write documents. The decision depends entirely on whether the ACL explicitly authorizes the principal's action on that resource.

Most ACL systems follow this basic pattern, though implementation details may vary:

- Some support both Allow and Deny entries, which may be evaluated sequentially or give precedence to deny rules.
- Others permit entries for groups or wildcards, providing broader permissions without listing every individual.
- Advanced implementations, such as Windows NTFS, support inheritance, where child resources or subordinate objects inherit permissions from parent folders or containers. This introduces a hierarchical structure to ACLs, enabling more efficient access management across nested resources.